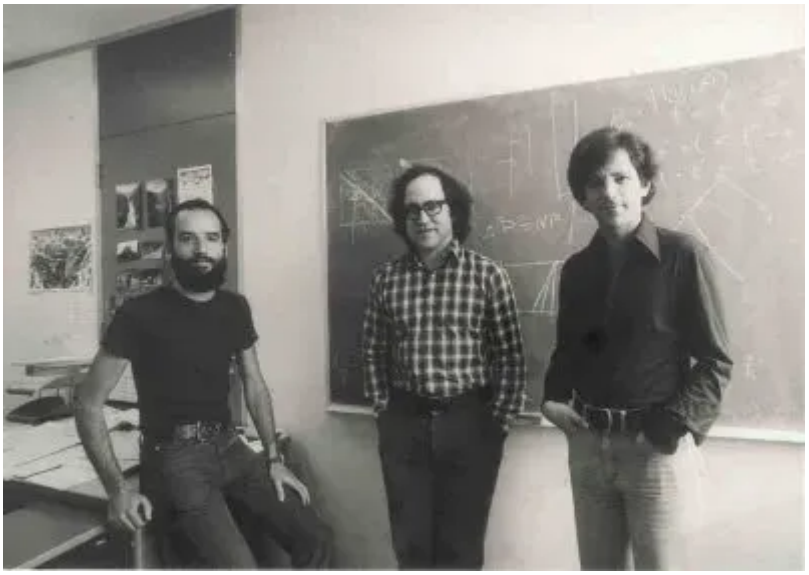
Medium

# A method to recover potential plaintexts when bad RSA keys were used.

4 min read · Aug 16, 2024

 g2f1 Follow



## Introduction

Hello everyone,

I hope you are all doing well. In this post, I will explain a method to recover possible plaintexts when incorrect RSA keys are used. Specifically, we will discuss a situation where d (the private key) cannot be calculated because the public exponent e is not coprime with phi(n), where phi(n) is the Euler's totient function of the modulus n. For brevity, I won't introduce the RSA algorithm here, but I invite you to check this Wikipedia page for more information: RSA (cryptosystem).

When gcd(e,phi(n)) > 1, we cannot compute the modular inverse of e, which is the private key used for decryption. As a result, decryption of the ciphertext is not straightforward. Fortunately, there is a method to recover all potential plaintexts.

Indeed, there are multiple solutions, and we will attempt to recover them all so you can manually determine the correct plaintext. This method works well but comes with some restrictions that I will detail in the following sections.

> *I recently read a paper by **Daniel Shumow** in which he describes a method for recovering potential plaintexts when incorrect RSA keys are used. However, his method is specifically applicable only when e is a prime number, e divides phi(n), and $e^2$ does not divide phi(n). When I encountered a case in a CTF challenge where e was not prime and e did not divide phi(n), i start thinking about a way to extend Shumow's method to address this scenario (even though the challenge itself had a simpler solution).*

## Mathematical Preliminary Concepts

Since the last paper of RSA , the group of units of the integers modolu n (Z/nZ)* noted Zn* is where all the calculation is performed ,although RSA works well for any integer betwen 1 and n. |G| represent the order of the finite group G ;it's the number of elements in G, and we we have this important result about finite abelian groups

Any abelian finite group $G$ such as $|G| = m \times n$ and $gcd(m, n) = 1$ is isomorph to $H \times K$ with $|H| = m$ and $|K| = n$. With math symbols :

$$G \cong H \times K$$

In our case Zn* can be decompose as :

$$\mathbb{Z}_n^* = E \times G \quad (1)$$

where $|E| = k$ , $|G| = \frac{\phi(n)}{k}$ and $k$ is an integer such as $gcd(e, \frac{\phi(n)}{k}) = 1$

We can calculate k explicitly by :

$$\prod_{d \in S} d^{\alpha}$$

where :

- $S = D^p_{\phi(n)} \cap D^p_e$
- $D^p_{\phi(n)} = \{p \setminus p \mid \phi(n) \text{ with } p \text{ prime } \} = D_{\phi(n)} \cap \mathbb{P}$
- $D^p_e = \{p \setminus p \mid e \text{ with } p \text{ prime } \} = D_e \cap \mathbb{P}$
- $\alpha$ is the power of the prime $p$ in the prime factorization of $\phi(n)$

But there is a simpler way to calculate k as you will see in the two algoritms below .

## Restrictions

- The first restriction is about the coprimality of $k$ and $\frac{\phi(n}{k}$ $\left( gcd(k, \frac{\phi(n}{k}) = 1 \right)$ because if it's not the case we can't decompose $\mathbb{Z}^*_n$ as $E \times G$. It is evident that this condition is always met.
- The second restriction is that $k$ must satisfy $k \leq e$ ensuring that $k \mid e$

## Why decryption is not a one to one function?

By (1), any element of Zn* can be decoposed as the product of two elements from E x G

$$\begin{aligned}
&\text{let } m, c \in \mathbb{Z}^*_n \text{ the plaintext and the ciphertext} \\
&\text{we have} \quad (1) \Longrightarrow \exists\ (g, l) \in G \times E\ :\ m = g \times l \\
&\textit{so} \quad c = m^e = (g \times l)^e = g^e \times l^e = g^e \bmod n \\
&\text{because} \quad \forall x \in E : x^k = 1\ \ (|E| = k)\ \ \text{and } k \mid e
\end{aligned}$$

So after the encryption we lost a part of our plaintext , it's about l , indeed there is

k potential plaintext (k possible value for l , because |E|=k)

### How can we recover all the potential plaintexts?

We first need to find integers $l$ such that $l^e \equiv 1 \ (mod\ n)$,there are called the $e^{th}$root of unity modulo $n$.We set an integer **max** to represent the number of roots we will generate , the roots are given by $r = a^{\frac{\phi(n)}{k}}$ for all integer $a \in$[[1,max]].
To demonstrate that $r$ are indeed $e^{th}$root of unity modulo $n$ :

$$\begin{aligned} r^e &= a^{e \times \frac{\phi(n)}{k}} \\ &= (g.l)^{e \times \frac{\phi(n)}{k}} \quad | \ (g,l) \in G \times E \\ &= \left(g^{\frac{\phi(n)}{k}}\right)^e \times (l^e)^{\frac{\phi(n)}{k}} \\ &= 1 \qquad because \ \ |G| = \frac{\phi(n)}{k} \ , \ |E| = k \end{aligned}$$

An algorithm to collect roots in the range of 1 to max ,after calculating k

**Algorithm 1** Find a set of length **max** of e-th roots of unity modulo $n$

**input**: max , $e$ the public exponent , $p$ and $q$ the prime used in RSA
$n \leftarrow p \times q$
$\phi(n) \leftarrow (p-1) \times (q-1)$
$k \leftarrow 1$
**while** $(gcd(\frac{\phi(n)}{k}, e)! = 1)$ **do**
    $k \leftarrow k \times gcd(\frac{\phi(n)}{k}, e)$
**end while**
$R \leftarrow \{\}$
**for** $(a \leftarrow 1$ **to** max$)$ **do**
    $r \leftarrow a^{\frac{\phi(n)}{k}} \ (mod\ n)$
    $R \leftarrow R \cup \{r\}$
**end for**
**return** $R$

After we move to determin the other part(statical part) of the potential plaintexts ,which is g

Since $gcd\left(e, \frac{\phi(n)}{k}\right) = 1 \quad \exists\, d \in \mathbb{N}$ such that $e \times d \equiv 1 \left(mod\ \frac{\phi(n)}{k}\right)$

So $c^d \equiv g^{ed} \equiv g \times g^{f \times \frac{\phi(n)}{k}} \equiv g\,(mod\ n)$

Now all we have to do is multiply each value of $l$ we found with $g$ and by doing so, we will obtain all the potential plaintexts..

An algorithm to describe this process is as follows:

**Algorithm 2** Determine $g$ and Recover all possible plaintexts

**input**: $e$ the public exponent , $p$ and $q$ , R the list of roots , c the ciphertext

$n \leftarrow p \times q$
$\phi(n) \leftarrow (p-1) \times (q-1)$
$k \leftarrow 1$
**while** $(gcd(\frac{\phi(n)}{k}, e)! = 1)$ **do**
    $k \leftarrow k \times gcd(\frac{\phi(n)}{k}, e)$
**end while**
$d \leftarrow e^{-1} \left(mod\ \frac{\phi(n)}{k}\right)$
$g \leftarrow c^d\,(mod\ n)$
$P \leftarrow \{\}$
**for** all $l \in R$ **do**
    $pt \leftarrow l \times g\,(mod\ n)$
    $P \leftarrow P \cup \{pt\}$
**end for**
**return** $P$

## Conclusion

What I have presented in this post is not a complete method for recovering the plaintext in real-world scenarios. The lack of information about the group **E** limits us to searching for **l** only as **e-th** roots of unity modulo **n**, which leaves some uncertainty about whether we have collected all possible values of **l** in **E** and, consequently, all possible plaintexts. This method may only be useful if the plaintext is in a readable format or the user of the algorithm has additional information about the original plaintext; by increasing the value of the variable **max** and checking each time for a match within the set **P** of potential plaintexts, there is a high probability that the original plaintext can successfully be recovered.I shared this method because it work well for me in the CTF challenge i already mention and because the math behind it is quite beautiful.

Rsa Algorithm | Group Theory | Rsa Keys | Cryptography

Follow

## Written by g2f1

2 followers · 1 following

Living to conquer CTF challenges.